

Pluggable Workflow Engine Support for Mifos X

Mifos Summer of Code 2026 — Final Report

Albin Sunny — MSoC 2026 contributor

21 September 2026

Contents

1. Project objective	1
2. Starting point and change of direction	2
3. Architecture	2
4. Work completed — Flowable	4
5. Work completed — three further engines	7
6. Work completed — the code generator	8
7. Defects found in shared Mifos libraries	9
8. Demonstration	10
9. Current state	12
10. Next steps	12
11. Links	13
12. Notes on accuracy	13

Contributor	Albin Sunny (@001Albin)
Mentor	Aleksandar Vidakovic (@vidakovic)
Upstream repository	openMF/mifos-workflow
Working branch	001Albin/mifos-workflow @ feature/flowable-usecases
Second repository	001Albin/mifos-conventions-gradle-suite @ feature/code-generator
Jira epic	MX-326
Report date	21 September 2026

1. Project objective

From the project description:

Create an extension mechanism that allows to integrate workflow engines. The main goal is to avoid leaking technology details of the specific workflow engine into Apache (R) Fineract. It should be possible to abstract the interactions with the workflow engines with a relatively simple Java interface.

A large amount of process logic in Apache Fineract is hard-coded as if/else branches inside business services. Every deploying institution gets the same sequence of steps, and changing it means changing Fineract itself.

The goal of this project is an abstraction layer that lets an institution choose its own workflow engine and define its own processes, without the choice of engine leaking into the banking code, and without the banking code being rewritten for each engine.

Because upstream acceptance could not be guaranteed, the work is contributed to openMF/mifos-workflow, a Mifos-controlled repository, rather than to Fineract core.

2. Starting point and change of direction

2.1 The original approach

The project began as a Maven multi-module SPI:

mifos-workflow-core/	WorkflowService – one interface, ~20 methods
mifos-workflow-adapter-flowable/	Flowable implementation
mifos-workflow-adapter-conductor/	Conductor placeholder
mifos-workflow-app/	Spring Boot application

Engine selection was to be made by a `WorkflowEngineFactory` at runtime.

Mid-term review raised a structural concern: the design required one `Flowable JavaDelegate` class per callable operation. The existing code already had sixteen such classes in `src/main/java/org/mifos/workflow/core/engine/` and exposing the full Fineract API through this mechanism would have required hundreds.

2.2 The restructure

The mentor rebuilt the project onto the Mifos build conventions — a set of Gradle convention plugins that standardise module layout, dependency versions, code quality tooling and licence headers across Mifos projects. The migration was merged upstream as commit 9c452fc on the dev branch, with authorship of the migration commit preserved to this contributor.

This changed the build system (Maven to Gradle), the module layout, the shape of the SPI, and the mechanism for engine callbacks. The conceptual work from the original approach — a neutral contract, engine isolation, shared Fineract logic, configuration-driven engine selection — carried over intact.

The remainder of the project was implementation and integration work on the new structure, which the mentor handed over untested:

“Should work, untested... will leave the rest for you.”

3. Architecture

3.1 Top-level layout

Every top-level module under `modules/` represents one problem domain:

<code>modules/</code>	
├─ <code>infrastructure/</code>	workflow engine integration – domain-agnostic
└─ <code>fineract/</code>	Fineract integration – engine-agnostic

Neither knows about the other. A future integration with an unrelated external system would become a third top-level module.

3.2 The four-role split

Almost every folder in the project divides the same four ways:

Folder	Contains	Visible to
core	interfaces, models, constants, properties	everyone
implementation	working code, package-private classes	nobody outside
mapping	MapStruct converters	its own module
starter	Spring auto-configuration	the application

Folder depth is the visibility signal: the deeper a class sits, the more private it is intended to be.

3.3 The SPI

modules/infrastructure/core defines the contract. Nine interfaces, each a single-method functional interface extending `MifosUsecase<Req, Res>` from the Mifos commons library:

Interface	Operation
<code>MifosFlowDeployUsecase</code>	install a workflow definition
<code>MifosFlowDeleteUsecase</code>	remove a deployed definition
<code>MifosFlowStartUsecase</code>	start a process instance
<code>MifosFlowCompleteUsecase</code>	complete a human task
<code>MifosFlowTerminateUsecase</code>	terminate a running instance
<code>MifosFlowTaskPendingUsecase</code>	list a user's pending tasks
<code>MifosFlowHistoryUsecase</code>	query completed instances
<code>MifosFlowSignalUsecase</code>	send an event into a waiting process
<code>MifosFlowReplayUsecase</code>	re-run a past process

Each has one paired request and response class. Requests and responses are never shared between operations — one request class corresponds to exactly one usecase, which is what makes the callback routing described in 3.5 possible.

3.4 Two directions

The distinction between usecase/ and support/ is the direction of the call:

infrastructure/usecase/<engine>	the application calls the engine "start this process", "terminate that one"
infrastructure/support/<engine>	the engine calls the application "I reached a service task, run the business logic"

Both exist for all seven engines: Cadence, Camunda, CIBSeven, Conductor, Eximee, Flowable, Operaton.

3.5 The universal delegate

This is the part that addresses the mid-term concern directly.

The old design, in `src/main/resources/processes/loan-origination.bpmn20.xml`:

```
<serviceTask id="createLoanInFineract" flowable:delegateExpression="${loanCreationDelegate}"/>
<serviceTask id="approveLoanInFineract" flowable:delegateExpression="${loanApprovalDelegate}"/>
<serviceTask id="rejectLoanInFineract" flowable:delegateExpression="${loanRejectionDelegate}"/>
```

Three service tasks, three delegate classes — sixteen across the six workflows, and unbounded as the API surface grows.

The new design uses one delegate for every operation, in every workflow, permanently:

```
public void execute(DelegateExecution execution) {
    var clazz = resolve(execution); // read the `type` field
    var request = jsonHelper.parseRequest(
        execution.getVariable(properties.getInputVariable()), clazz); // rebuild the request
    var response = registry.execute(request); // look up and run the
        ⇨ usecase
    store(execution, response); // write the result back
}
```

The BPMN supplies the request class name in a field; because one request class maps to exactly one usecase, the request object is its own routing key. Adding a new operation means writing a request class and a usecase

— work that would be done anyway — and referencing the type name in the BPMN. The delegate is never modified.

3.6 Engine selection

Every engine implementation carries:

```
@ConditionalOnBooleanProperty(FLOWABLE_WORKFLOW_PROPERTIES_ENABLED)
```

which resolves to `mifos.workflow.infrastructure.flowable.enabled`. When false, Spring does not create those beans. Controllers depend only on the interfaces, so switching engines is a configuration change with no code change and no recompilation.

The design is deliberately one engine per application instance. The mentor's reasoning, from a project meeting: allowing several engines in one instance eventually requires workflows in different engines to signal each other, which is difficult to reason about and to operate.

3.7 Transport

`modules/infrastructure/transport/rest` exposes nine endpoints, one per operation:

```
POST /workflows/deploy          POST /workflows/history
POST /workflows/delete          POST /workflows/signal
POST /workflows/start           POST /workflows/replay
POST /workflows/complete        POST /workflows/tasks/pending
POST /workflows/terminate
```

All use the versioned vendor media type `application/vnd.mifos.workflow+json; charset=UTF-8; version=1.0`, which allows future API versions to coexist on the same paths.

The module is named `transport` rather than `rest` because `gRPC` and `RSocket` are possible siblings later.

4. Work completed — Flowable

4.1 Usecase implementations

All nine Flowable usecases are implemented. All were previously present as stubs that called the engine and discarded the result, or had no body at all.

Usecase	Before	After
Deploy	engine call, result discarded	returns the deployment id
Start	engine call, result discarded	returns the process instance id
Complete	engine call, empty response	returns the completed task id
Terminate	engine call, empty response	returns the terminated process id
Delete	engine call, empty response	returns the deleted deployment id
TaskPending	query run, mapper injected but unused	returns the mapped task list
History	query run, result discarded	returns the mapped history list, filtered by id when supplied
Signal	no body — request lacked the required fields	delivers a named signal, globally or to one instance
Replay	no body — no native engine operation exists	rebuilds a finished process from history and starts it again

Example — `FlowableFlowDeployUsecase` before:

```
// var deployment =
repositoryService.createDeployment()
    .addInputStream(request.getName(), request.getProcessDefinition())
    .name(request.getName())
    .deploy();

// TODO: return some sensible data
return MifosFlowDeployResponse.builder().build();
```

and after:

```
var deployment = repositoryService.createDeployment()
    .addInputStream(request.getName(),
        new ByteArrayInputStream(request.getProcessDefinition().getBytes(StandardCharsets.UTF_8)))
    .name(request.getName())
    .deploy();

log.debug("deployed process definition {} with id {}", request.getName(), deployment.getId());

return MifosFlowDeployResponse.builder()
    .id(UUID.fromString(deployment.getId()))
    .build();
```

4.2 Signal and Replay

These two were blocked for most of the project because the request models did not carry enough information to call the engine at all. Both were unblocked after the design question was settled with the mentor.

Signal. MifosFlowSignalRequest carried only UUID id. Flowable’s signalEventReceived needs a signal *name*, and delivering to one specific instance additionally needs that instance’s execution id. Two fields were added:

```
private String signalName;
private Map<String, Object> variables;
```

The usecase supports both shapes of the operation. With no id, the signal is broadcast to every process waiting on that name; with an id, the matching execution is looked up first and the signal delivered only there:

```
if (request.getId() == null) {
    runtimeService.signalEventReceived(request.getSignalName(), variables);
} else {
    var execution = runtimeService.createExecutionQuery()
        .processInstanceId(request.getId().toString())
        .signalEventSubscriptionName(request.getSignalName())
        .singleResult();
    runtimeService.signalEventReceived(request.getSignalName(), execution.getId(), variables);
}
```

Replay. Flowable has no replay operation, so what “replay” means had to be decided rather than looked up. The decision taken, and agreed with the mentor, is: a replay reads a finished instance out of history, recovers the variables it ran with, and starts a brand new instance of the same process definition with those variables — optionally overridden by any variables supplied on the request. The original instance is untouched; replay is not a rollback.

```
var source = historyService.createHistoricProcessInstanceQuery()
    .processInstanceId(sourceId).singleResult();

Map<String, Object> variables = new HashMap<>();
historyService.createHistoricVariableInstanceQuery()
    .processInstanceId(sourceId).list()
    .forEach(v -> variables.put(v.getVariableName(), v.getValue()));

if (request.getVariables() != null) {
    variables.putAll(request.getVariables());
}
```

```
var replayed = runtimeService.startProcessInstanceByKey(source.getProcessDefinitionKey(), variables);
```

Replay was verified end to end through REST. A finished instance 23cff52a-a6f7-11f1-8b6d-4ccf7cb90771 was replayed with a request carrying no variables at all, and the new instance 672bc345-a6f7-11f1-8b6d-4ccf7cb90771 came up already holding "assignee": "rajesh" — a value that appears nowhere in the request and could only have come from the original run's history. That is the proof that variable recovery works, rather than a fresh empty process being started.

4.3 The Void decision for zero-parameter operations

Some generated Fineract operations take no parameters at all. The open question was whether to generate an empty request class for them anyway, for symmetry.

The mentor's decision was to skip the class entirely and use the JDK's own Void type:

```
public interface FineractSomethingUsecase extends MifosUsecase<Void, FineractSomethingResponse> { }

FineractSomethingResponse execute(Void request);
```

This removes a whole family of empty classes that would have existed only to be ignored. The generator described in section 6 applies this rule.

4.4 Contract changes

Three changes were made to shared models in infrastructure/core, on top of the two Signal/Replay additions above. Each was driven by a runtime failure or a structural mismatch rather than preference, and each was raised with the mentor.

MifosFlowDeployRequest.processDefinition: InputStream → String

Jackson cannot deserialise an InputStream from a JSON body. The endpoint returned:

```
HttpMessageConversionException: Type definition error: [simple type, class java.io.InputStream]
(through reference chain: MifosFlowDeployRequest["processDefinition"])
```

The field now carries the definition as text and is converted to a stream inside the usecase. This affects all seven engines equally — the same limitation would have applied to each.

MifosFlowTaskPendingResponse: single task → List<MifosFlowTask>

The response held nine fields describing one task, while the query is `taskService.createTaskQuery().taskAssignee(...).list()` which returns many. The injected `FlowableTaskPendingMapper` could not be used because there was no list to map into. A new `MifosFlowTask` model holds the per-task fields; the response holds a list of them; the mapper gained a list method.

MifosFlowHistoryResponse: UUID id → List<MifosFlowHistoryEntry>

The same mismatch. A new `MifosFlowHistoryEntry` model was added, along with a new `FlowableHistoryMapper`. The usecase now also honours `request.getId()`, filtering to a single instance when one is supplied and returning all completed instances when it is not — previously the request was ignored entirely.

4.5 Error handling

`MifosFlowException.MifosFlowErrorCode` contained two entries. Seven were added, each corresponding to a failure mode of the implemented operations:

```
MIFOS_FLOW_ERROR_DEPLOYMENT_FAILED
MIFOS_FLOW_ERROR_DEPLOYMENT_NOT_FOUND
MIFOS_FLOW_ERROR_PROCESS_DEFINITION_NOT_FOUND
MIFOS_FLOW_ERROR_PROCESS_NOT_FOUND
MIFOS_FLOW_ERROR_PROCESS_START_FAILED
MIFOS_FLOW_ERROR_TASK_NOT_FOUND
MIFOS_FLOW_ERROR_ENGINE_UNAVAILABLE
```

Numbering follows the existing scheme, derived from `ERROR_CODE_START` and `ERROR_CODE_INCREMENT` rather than hard-coded.

The six `messages*.properties` resource bundles in `infrastructure/core` were empty. All are now populated with one entry per error code, in English, German, Spanish, French and Indonesian. The non-English translations are machine quality and should be reviewed by native speakers.

The codes are defined but not yet thrown from the usecases; wiring them in awaits confirmation of the intended pattern, as the commons library provides its own `MifosErrorHandlingControllerAdvice`.

5. Work completed — three further engines

With Flowable complete, the same nine operations were implemented for **CIBSeven**, **Operaton** and **EximeeBPMS**, in the order the mentor asked for. That is 27 further usecase classes and 6 further mappers.

5.1 Why these three, and why they were feasible in the time available

All three are forks of Camunda 7 and share its engine API almost exactly. The service interfaces — `RuntimeService`, `TaskService`, `RepositoryService`, `HistoryService` — and their method signatures line up across all three; what differs is the package root:

```
org.cibseven.bpm.engine.*
org.operaton.bpm.engine.*
org.eximeebpms.bpm.engine.*
```

That made the work tractable, but the code still had to be read and adjusted rather than copied blindly. Two classes of real difference had to be handled.

5.2 Non-UUID identifiers

Flowable hands back identifiers that happen to parse as UUIDs, so `UUID.fromString(...)` works. The Camunda-lineage engines do not guarantee that — their ids are engine-generated strings and may be of any shape. `UUID.fromString` on such an id throws `IllegalArgumentException` at runtime, which would have surfaced only in production.

A small utility per engine converts safely, falling back to a stable derived UUID rather than throwing:

```
@UtilityClass
class CibsevenIds {
    static UUID toUuid(String id) {
        if (id == null) return null;
        try {
            return UUID.fromString(id);
        } catch (IllegalArgumentException e) {
            return UUID.nameUUIDFromBytes(id.getBytes(StandardCharsets.UTF_8));
        }
    }
}
```

The fallback is deterministic — the same engine id always yields the same UUID — so ids remain stable across calls and restarts.

5.3 Two missing build dependencies

`modules/infrastructure/usecase/operaton/mapping/build.gradle` and the Eximee equivalent were both empty files. The CIBSeven one had been set up by the mentor; the other two had not, so the mapper modules could not see the engine's model types at all and `MapStruct` had nothing to generate against. Both now declare the engine's Spring Boot starter:

```
dependencies {
    api "org.operaton.bpm.springboot:operaton-bpm-spring-boot-starter"
}
```

This was a genuine gap in the project skeleton rather than a mistake introduced by this work, and it would have blocked anyone picking up either engine.

5.4 Verification status — stated plainly

The three new engines **compile**. `gradlew build -x test` completes with exit code 0 across the whole project with all of them present.

They have **not** been exercised at runtime. Flowable was driven through Postman operation by operation; CIBSeven, Operaton and Eximee have not been, because each needs its own engine instance and datasource configured, and the time remaining did not allow it. This report does not claim more than was done: Flowable is runtime-verified, the other three are compile-verified.

6. Work completed — the code generator

The second half of the project moved to a separate repository, `mifos-conventions-gradle-suite`, on the mentor's instruction.

6.1 The problem it solves

The Fineract REST SDK is generated from `fineract.yml`, an OpenAPI document of roughly 64,000 lines, producing 148 `*Api.java` files with around twenty operations each. For each of those operations, the Mifos side needs five hand-written files:

<code><Op>Request.java</code>	the neutral request model
<code><Op>Response.java</code>	the neutral response model
<code><Op>Usecase.java</code>	the interface
<code><Op>UsecaseImpl.java</code>	the implementation that calls the SDK
<code><Op>Mapper.java</code>	the MapStruct converter between the two

At roughly 3,000 operations that is around 15,000 files. Writing them by hand is not realistic, and hand-writing them once means hand-maintaining them every time Fineract's API changes.

6.2 How it works

The generator reads the already-generated SDK source as data using **JavaParser** — `StaticJavaParser.parse()` turns a `.java` file into a tree of objects, and a `CompilationUnit` is used to build and print the new files. Parsing is not compiling: the input does not need its imports to resolve, which is what makes it possible to point the generator at SDK source without building the entire SDK first.

For each method on each `*Api.java` class the generator determines the operation name, the return type (unwrapping `ResponseEntity<...>` and `Optional<...>`), and the parameters, distinguishing a `@RequestBody` from path and query parameters. It then emits the five files.

The parameter distinction matters because the two kinds are called differently in the generated implementation:

```
var args = method.getParameters().stream()
    .map(p -> p.isAnnotationPresent("RequestBody")
        ? "mapper.map(request)" // body → mapped object
        : "request.get" + capitalize(p.getNameAsString()) + "()") // path/query → plain getter
    .collect(Collectors.joining(", "));
```

The generator loops over a folder rather than a hard-coded list of classes, so adding a new API file to the input requires no change to the generator.

6.3 Result

Run against `ClientApi.java` and `CashiersApi.java` together, the generator produced **79 files** with no name collisions between the two APIs and no duplicated mapper methods.

6.4 Bugs found by testing against real input

The generator worked on the first small example and then failed repeatedly on real SDK files. Each failure was a real gap, and each is fixed:

Problem	Fix
Path and query parameters were being mapped as though they were request bodies	parameter kind is now detected from the <code>@RequestBody</code> annotation
The same mapper method was emitted twice when two operations shared a model	mapper methods are de-duplicated before printing
Two APIs with a similarly named operation produced two classes with the same name	operation names are derived so that they stay distinct across API files
<code>List<...></code> , <code>Set<...></code> and <code>Collection<...></code> return types produced malformed class names	collection types are flattened to their element type
Built-in types such as <code>String</code> and <code>Long</code> were being given nonsense imports	a built-in list is excluded from import generation
JDK types such as <code>LocalDate</code> and model types were referenced without imports	an explicit JDK type map plus model-package imports

This is the part of the project that most clearly benefited from being tested against the real thing rather than a toy input.

6.5 Limits

The generated output has been inspected but not compiled as part of a build. Nested model imports — a model that references another model that references a third — are not fully resolved. Only `List`, `Set` and `Collection` are recognised as collection types. These are recorded here as known gaps, not as finished work.

7. Defects found in shared Mifos libraries

Integration work surfaced five defects. They are recorded here because each affects any project built on the same libraries, not only this one.

7.1 Request filter never continues the chain

`MifosContextRequestFilter` in `mifos-commons-boot`:

```
@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain
    ↪ filterChain)
    throws ServletException, IOException {
    // TODO: implement this!
}
```

A servlet filter must call `filterChain.doFilter(request, response)` to pass the request onward. This one does not, so every request to any application using the library terminates in the filter and returns an empty HTTP 200 — including Spring's own endpoints.

This was the cause of a symptom that took considerable time to isolate: every URL, valid or invalid, returned 200 with `Content-Length: 0`. Adding the `doFilter` call locally and republishing produced a correct 404 on an unknown route and correct routing to the controllers.

This is the most consequential finding of the project. Any Spring MVC application depending on `mifos-commons-boot-transport-rest-imperative` is currently unable to serve any request.

7.2 Application does not depend on the workflow modules

modules/fineract/application/backend/implementation did not have infrastructure-starter on its runtime classpath. Verified with:

```
gradlew :mifos-workflow-fineract-application-backend-implementation:dependencies --configuration runtimeClasspath
```

which returned no infrastructure entries at all. None of the controllers, usecases, support classes or the engine were present in the running application.

7.3 Engine auto-configurations activate unconditionally

With the infrastructure starter present, all seven engines land on the classpath. Four of them ship Spring auto-configurations that activate without checking whether that engine is wanted, and fail at startup looking for a server that is not running:

```
io.orkes.conductor.client.spring.OrkesConductorClientAutoConfiguration
org.cibseven.bpm.spring.boot.starter.CamundaBpmAutoConfiguration
org.operaton.bpm.spring.boot.starter.OperatonBpmAutoConfiguration
com.netflix.conductor.client.spring.ConductorClientAutoConfiguration
```

The Mifos classes are correctly guarded with `@ConditionalOnBooleanProperty`; these third-party ones are not. An aggregating starter that includes all engines cannot currently be used without excluding them.

7.4 SDK starter declares no dependencies

mifos-commons-oas-imperative-rest-sdk-fineract-starter is published as `<packaging>pom</packaging>` with no `<dependencies>` block — only licence, developer and SCM metadata. A starter of this kind exists to bring in other artifacts; this one brings in nothing, so `api "...-fineract-starter"` in the consuming module supplies no classes.

Confirmed by building the artifact from source: the locally produced pom is identical to the published one, so this is not a caching or publication problem. It was confirmed separately that the OAS artifacts are not published to JFrog at all — hiding the local `.m2/repository/org/mifos` directory and rebuilding caused resolution to fail, proving that the build depends on locally published artifacts rather than remote ones.

7.5 OAS repository does not build with its documented command

The README for mifos-commons-oas-imperative gives:

```
./gradlew mifosConfigUnzip
./gradlew clean build
./gradlew clean build publish publishToMavenLocal
```

The second and third fail at `sonarlintMain` on a commented-out block in `FineractRestSdkConfiguration.java:32`. `publishToMavenLocal` on its own succeeds and produces the required artifacts.

8. Demonstration

Verified end to end through Postman against a locally running instance, using Flowable.

8.1 Running the application

```
gradlew :mifos-workflow-fineract-application-backend-implementation:bootRun \
  --args="--mifos.workflow.infrastructure.flowable.enabled=true"
```

Local workarounds required, none of which are committed:

- infrastructure-starter added as a dependency of the application module (7.2)
- the four engine auto-configurations excluded in `application.yml` (7.3)
- `@ComponentScan` widened on `Main` so the transport and Flowable usecase packages are picked up
- `filterChain.doFilter(...)` added to `MifosContextRequestFilter` in a local build of mifos-commons-boot (7.1)

- `bootRun { systemProperty 'java.io.tmpdir', ... }` — local only, working around a Windows machine whose `java.io.tmpdir` resolves to an unwritable directory

The workflow used is `loan-origination.bpmn20.xml` from the legacy `src/` tree, with the process id renamed to `loan-demo` and the three `flowable:delegateExpression` attributes replaced, since the sixteen delegate beans they reference no longer exist. A second definition with a signal catch event was written specifically to exercise the Signal operation.

8.2 Sequence

All requests are POST with `Content-Type: application/vnd.mifos.workflow+json; charset=UTF-8; version=1.0`. The vendor media type is required — sending `application/json` returns 415 Unsupported Media Type.

Deploy — `/workflows/deploy`, body containing the definition name and the BPMN XML:

```
{ "id": "1adacc5f-9efc-11f1-8bbe-4ccf7cb90771" }
```

Start — `/workflows/start`:

```
{ "key": "loan-demo",
  "variables": { "loanOfficer": "rajesh", "assignee": "rajesh", "approver": "manager" } }

{ "id": "0ba5cadb-9f0a-11f1-bc76-4ccf7cb90771" }
```

Pending tasks — `/workflows/tasks/pending`, { `"userId": "rajesh"` }:

```
{ "tasks": [ {
  "taskId": "0bacf6d3-9f0a-11f1-bc76-4ccf7cb90771",
  "name": "Submit Loan Application",
  "processId": "0ba5cadb-9f0a-11f1-bc76-4ccf7cb90771",
  "processDefinitionId": "loan-demo:1:08cbf6ea-9f0a-11f1-bc76-4ccf7cb90771",
  "assignee": "rajesh",
  "createTime": "2026-08-23T15:48:11.933",
  "description": "Loan officer submits the loan application",
  "priority": 50
} ] }
```

Complete — `/workflows/complete` with that task id:

```
{ "id": "0bacf6d3-9f0a-11f1-bc76-4ccf7cb90771" }
```

Pending tasks again — the same process instance, now at the next step:

```
{ "tasks": [ {
  "taskId": "1b98481b-9f0a-11f1-bc76-4ccf7cb90771",
  "name": "Review Loan Application",
  "processId": "0ba5cadb-9f0a-11f1-bc76-4ccf7cb90771",
  "createTime": "2026-08-23T15:48:38.665",
  "description": "Review the loan application for completeness and accuracy"
} ] }
```

The unchanged `processId` and the changed task confirm the instance advanced rather than a new one being created.

Replay — `/workflows/replay`, given only the id of a finished instance and no variables:

```
{ "sourceProcessId": "23cff52a-a6f7-11f1-8b6d-4ccf7cb90771" }

{ "id": "672bc345-a6f7-11f1-8b6d-4ccf7cb90771" }
```

Querying the new instance shows `"assignee": "rajesh"` present, recovered from the original run's history rather than supplied on the request.

Terminate, **history** and **delete** were verified in the same way. History returns entries only for completed instances, correctly filtered when an id is supplied and empty for an unknown id. After deleting the deployment, a further start request fails with `No process definition found for key 'loan-demo'`, confirming removal.

8.3 What this does and does not show

Demonstrated: the outbound direction of the SPI — the application driving the engine through a neutral, engine-agnostic API, with no Flowable type appearing in any request or response. Deploy, start, complete, terminate, delete, pending tasks, history and replay were all confirmed against live responses.

Not fully demonstrated:

- **The Signal wake-up sequence.** The usecase is implemented and a BPMN definition with a signal catch event was prepared, but the run was interrupted by the 415 media-type error described above and completion of the sequence was not confirmed afterwards. It is listed as unverified rather than as working.
 - **The inbound direction.** FlowableMifosUsecaseDelegate is present and wired, but showing a BPMN service task calling back into a Fineract usecase requires the support package in the component scan and a working MifosUsecaseRegistry bean, which is not yet resolved.
 - **CIBSeven, Operaton and Eximee at runtime**, per section 5.4.
-

9. Current state

Working and verified

- Full Gradle build green across all modules, build -x test exit code 0
- Application starts; Flowable initialises and creates its schema
- All nine Flowable usecases implemented; eight verified through REST, Signal implemented but its wake-up sequence unverified
- Twenty-seven further usecases and six mappers for CIBSeven, Operaton and Eximee — compile-verified
- Nine error codes defined; six message bundles populated
- Code generator producing all five file types, 79 files from two real SDK API classes
- One defect fixed in mifos-commons-boot; two missing build dependencies added

Not complete

- Signal's wake-up sequence not confirmed end to end.
 - CIBSeven, Operaton and Eximee not exercised at runtime.
 - Camunda 8, Conductor and Cadence remain stubs. These are not Camunda 7 forks — Camunda 8 uses Zeebe over gRPC, and Conductor and Cadence are different architectures entirely — so the copy-and-adapt approach that worked for the other three does not apply. Each needs its nine usecases written against its own SDK.
 - The generated code from the generator has not been compiled as part of a build; nested model imports are unresolved.
 - Error codes are defined but not yet raised from the usecases.
 - The ClientApi bean in FineractRestSdkConfiguration.java is commented out, so nothing currently reaches a live Fineract instance.
 - Persistence is in-memory H2, because no datasource is configured and Spring Boot falls back to it. State is lost on restart. .mifos/config/compose/postgresql.yml and .mifos/config/env/postgres.env exist but are not wired into the application.
 - The six BPMN definitions exist only in the legacy src/ tree and have no home in the new module structure.
 - The doc modules are placeholders.
-

10. Next steps

For whoever continues this work

1. Run the Signal wake-up sequence to completion with the correct vendor media type, and record it.
2. Stand up CIBSeven, Operaton and Eximee with their own datasources and drive all nine operations through Postman, as was done for Flowable.
3. Compile the generator's output as part of a build, and resolve nested model imports.

4. Wire the error codes into the usecases, following whatever pattern the commons `MifosErrorHandlingControllerAdvice` expects.
5. Configure PostgreSQL using the compose and environment files already present in the Mifos config.
6. Demonstrate the inbound direction — a BPMN service task calling a Fineract usecase through the universal delegate.
7. Implement Camunda 8, Conductor and Cadence, which need genuinely different client code.
8. Decide where the BPMN definitions belong in the new structure, and write the module documentation in the doc modules.

For the community

1. **MifosContextRequestFilter (7.1) needs fixing upstream.** Until it is, no Spring MVC application using `mifos-commons-boot-transport-rest-imperative` can serve a request.
2. **The SDK starter (7.4) needs a dependencies block,** or consuming modules must depend on the implementation artifact directly. The OAS artifacts also need publishing to JFrog if consumers are not to be required to build them locally.
3. **The aggregating infrastructure starter (7.3) cannot currently include all engines,** because four third-party auto-configurations activate unconditionally. Either the starter should be split per engine, or the exclusions should be applied centrally.
4. **The remaining three engine adapters** — Camunda 8, Conductor, Cadence — are open, well-defined pieces of work. The contract exists and four worked examples now exist alongside it.
5. **BPMN definitions should be made overridable** so an institution can customise a process by supplying its own definition rather than modifying the project, which was the mentor's stated intent for the design.

11. Links

Upstream repository	https://github.com/openMF/mifos-workflow
Working branch	https://github.com/001Albin/mifos-workflow/tree/feature/flowable-usecases
Code generator branch	https://github.com/001Albin/mifos-conventions-gradle-suite/tree/feature/code-generator
Migration PR (merged upstream)	https://github.com/openMF/mifos-workflow/pull/74
Mifos commons (Spring Boot)	https://github.com/monkeysintown/mifos-commons-boot
Mifos build conventions (base)	https://github.com/monkeysintown/mifos-conventions-gradle-base
Mifos build conventions (boot)	https://github.com/monkeysintown/mifos-conventions-gradle-boot
Generated REST SDKs	https://github.com/monkeysintown/mifos-commons-oas-imperative

12. Notes on accuracy

Everything in this report is drawn from source read directly, from build and application output observed locally, or from HTTP responses captured during testing.

Where a change touches a shared contract, that is stated along with the failure that prompted it. Where something is unverified it is recorded as unverified rather than described as done — in particular the Signal wake-up sequence, the runtime behaviour of CIBSeven, Operaton and Eximee, the inbound delegate path, and the compilation of the generator's output. The distinction between "it compiles" and "it runs" is kept explicit throughout, because they are not the same claim.